

SIMATS ENGINEERING
(SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – ARCHITECTURE DESIGN ASSIGNMENT

Course Code: CSA10

Course Title: Software Engineering

CO Assessed: CO2

Assessment: Assessment Tool 2 – Architecture Design Assignment

Weightage: 20% Total Marks: 25

CO2 Statement: *Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills.*

Student Name: G.pradeep ranga reddy

Reg. No.: 192311289

Date: 14-08-2026

Duration: 60 minutes

Assigned Problem Statement:

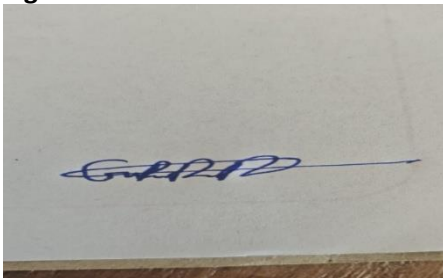
Intelligent Disaster Early Warning and Evacuation Management System

Code of Conduct

I G.pradeep ranga reddy 192311289

(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

A photograph of a white sheet of paper with a handwritten signature in blue ink. The signature is stylized and appears to read 'G. Pradeep Ranga Reddy'.

1. Analyze System Requirements

1.1 Problem Statement & System Objectives

The assigned problem statement is the “Intelligent Disaster Early Warning and Evacuation Management System (IDEWEMS).” The system aims to detect natural and man-made disasters (floods, earthquakes, cyclones, tsunamis, wildfires) as early as possible using sensor, satellite and citizen-reported data, assess risk in real time, and coordinate a rapid, life-saving response. The core objectives are:

- Continuously monitor environmental and infrastructural data to detect early indicators of a disaster.
- Predict the type, location and severity of an impending disaster using analytical/ML models.
- Disseminate timely, accurate alerts to citizens, first responders and government authorities through multiple channels.
- Guide affected populations to safety through dynamically computed evacuation routes and shelter allocation.
- Provide authorities a unified situational-awareness dashboard to coordinate the overall response.

1.2 Functional Requirements

The functional requirements describe the concrete capabilities the system must provide:

Functional Requirement	Description
Real-Time Data Ingestion	Continuously acquire data streams from IoT sensors, seismographs, weather satellites, drones/CCTV and social media feeds.
Disaster Prediction & Risk Scoring	Apply ML/statistical models on incoming data to predict disaster likelihood, type and severity, and generate a risk score.
Multi-Channel Alert Dissemination	Issue alerts through SMS, IVR calls, push notifications, sirens/public address systems and social media simultaneously.
Evacuation Route Computation	Compute and update safe evacuation routes using live hazard, road-closure and traffic data.
Shelter & Resource Management	Track shelter capacity, medical supplies, food and manpower, and allocate resources to affected zones.
Authority Dashboard	Provide disaster management authorities a real-time situational-awareness dashboard with maps, analytics and control actions.
Citizen Mobile Application	Allow citizens to receive alerts, mark safety status, view evacuation routes and report incidents.
External System Integration	Interface with government disaster-authority systems, telecom gateways and emergency services (police/fire/ambulance).
Historical Logging & Reporting	Store historical event and response data for post-disaster analysis and continuous model improvement.

1.3 Non-Functional Requirements

Because the system operates in life-critical, high-stress scenarios, the following non-functional requirements strongly influence the architecture:

Non-Functional Requirement	Description
Scalability	System must scale elastically to handle sudden spikes (millions of concurrent users) during an active disaster.
Availability	Target near-continuous uptime (24×7×365) since the system is life-critical; must tolerate partial infrastructure damage.
Reliability	Alerts and predictions must be dependable and fault-tolerant, with no single point of failure.
Performance	End-to-end alert dissemination latency must be in the order of a few seconds from detection to citizen notification.
Security	Protect sensitive location and personal data; prevent spoofed/false alerts through authentication and validation.
Maintainability	Individual components (e.g., ML models, notification channels) must be upgradable without affecting the whole system.
Interoperability	Must integrate with heterogeneous external systems using standard protocols (REST, MQTT, webhooks).
Usability & Accessibility	Interfaces must be multilingual, low-bandwidth friendly and accessible to differently-abled users.

1.4 Key Quality Attributes Influencing the Architecture

Among the general non-functional requirements, six quality attributes most directly shape the architectural decisions for this system:

Quality Attribute	Relevance to This System
Scalability	High priority — event volume and user load can grow 100× within minutes of a disaster onset.
Availability	Critical — the system is life-safety software and cannot have prolonged downtime.
Reliability	Critical — false negatives (missed alerts) can cost lives; false positives erode public trust.
Performance	High priority — alerts must reach citizens within seconds of risk detection.
Security	High priority — must prevent spoofed alerts, protect citizen location data and secure authority controls.
Maintainability	Medium-high priority — prediction models and channels evolve frequently and need independent updates.

2. Select a Suitable Software Architecture

2.1 Selected Architectural Style

A Hybrid Architecture combining Microservices and Event-Driven Architecture (EDA), layered internally within each service and supplemented with Edge Computing for localized sensor pre-processing, is proposed for the Intelligent Disaster Early Warning and Evacuation Management System.

2.2 Justification

- **Continuous, heterogeneous data streams:** Sensor, satellite, social-media and CCTV data arrive asynchronously and unpredictably. An event-driven backbone (publish/subscribe over a message broker) naturally models this “detect → react” nature better than a request/response-only style.
- **Independent, uneven scaling needs:** During an active disaster, the Alert and Evacuation-Routing services may need to scale a hundredfold in minutes, while the User-Management service load stays flat. Microservices allow each capability to be deployed, scaled and monitored independently.
- **Fault isolation:** A failure in a single channel (e.g., the SMS gateway) or a single service must not bring down the whole warning system. Loosely-coupled microservices communicating over an event bus contain failures to one component.
- **Low-latency local response:** Disaster sites often have unreliable connectivity. Edge nodes perform local filtering and anomaly detection close to the sensors, so critical decisions do not always depend on a round trip to the cloud.
- **Internal maintainability:** Within each microservice, a classic layered structure (presentation / business-logic / data-access) keeps the codebase organised, testable and easy for new team members to maintain.
- **Extensibility:** New data sources (e.g., a new sensor network) or new alert channels can be added as new services/subscribers without modifying existing services, supporting long-term evolution of the system.

A pure monolithic layered architecture was considered but rejected because it could not scale individual hot-spots independently and would create a single point of failure unacceptable for a life-safety system. A pure serverless approach was considered for the alert-dissemination functions (and is, in fact, used selectively for burst-driven notification fan-out) but was not chosen as the sole style because long-running ML prediction workloads and stateful shelter/resource tracking are better suited to persistently running microservices.

3. Software Architecture Diagram

The diagram below illustrates the overall structure of the proposed system: data sources, edge pre-processing, the ingestion/event-bus layer, the microservices business layer, persistent data stores, the multi-channel dissemination layer, client applications and external systems, along with the primary communication paths between them.

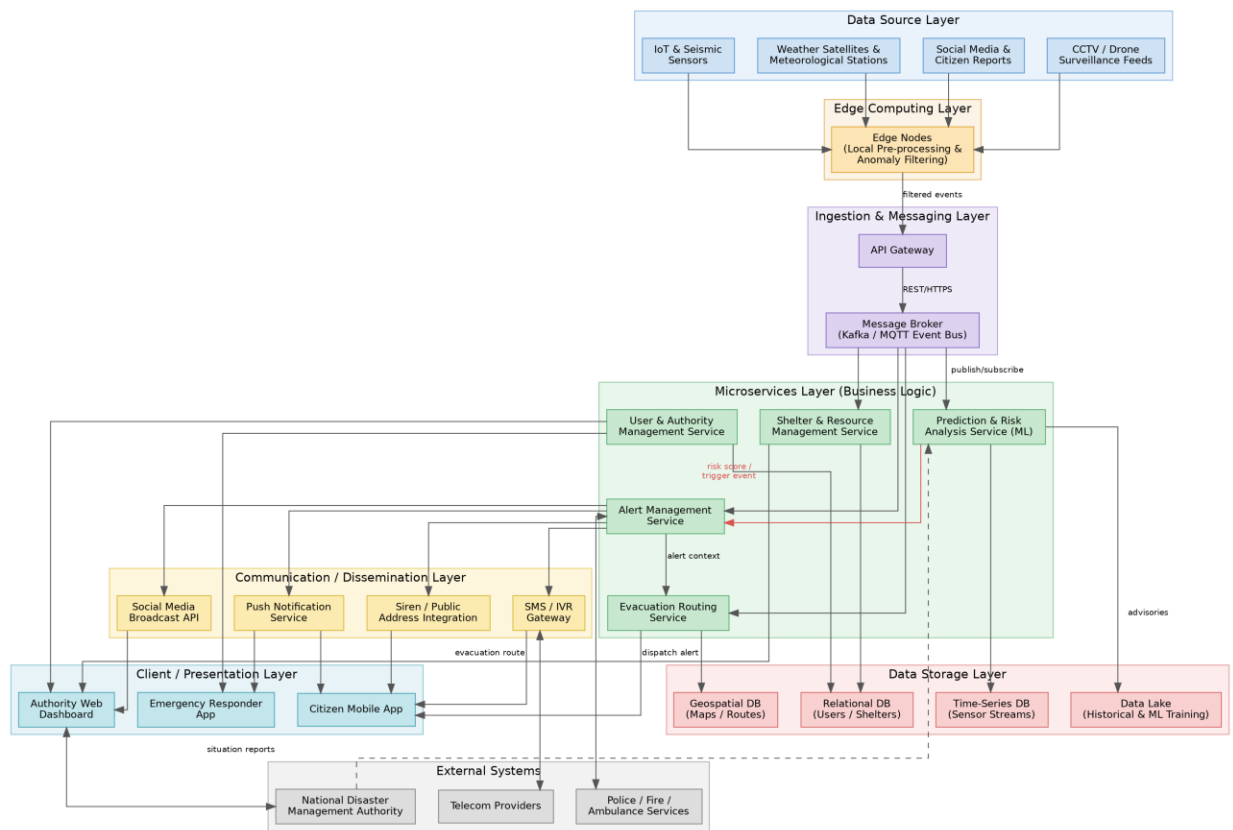


Figure 1: Architecture of the Intelligent Disaster Early Warning and Evacuation Management System

4. Explain Components and Their Interactions

4.1 Component Responsibilities

Layer	Key Components	Responsibility
Data Source Layer	IoT/seismic sensors, weather satellites, social media, CCTV/drones	Originates raw observational data used for disaster detection.
Edge Computing Layer	Edge nodes	Performs local pre-processing and anomaly filtering close to the data source, reducing bandwidth use and latency before forwarding relevant events.
Ingestion & Messaging Layer	API Gateway, Message Broker (Kafka/MQTT)	Authenticates and routes incoming data through a single entry point; the broker decouples producers from consumers via publish/subscribe events.
Microservices Layer	Prediction & Risk Analysis, Alert Management, Evacuation Routing, Shelter & Resource Management, User & Authority Management	Implements the core business logic — risk prediction, alert triggering, route computation and resource coordination — each independently deployable and scalable.

Layer	Key Components	Responsibility
Data Storage Layer	Time-series DB, Geospatial DB, Relational DB, Data Lake	Persists sensor streams, map/route data, user/shelter records and historical data for analytics and ML training, using the database best suited to each data shape (polyglot persistence).
Communication/Dissemination Layer	SMS/IVR gateway, push notifications, siren/PA integration, social media broadcast	Fans a single alert event out across multiple external channels so citizens are reached through whichever channel is available to them.
Client/Presentation Layer	Citizen mobile app, authority web dashboard, emergency responder app	Presents alerts, routes and situational data to end users through role-appropriate interfaces.
External Systems	National disaster authority, telecom providers, police/fire/ambulance	Government and third-party systems that the platform must interoperate with for advisories, message delivery and emergency dispatch.

4.2 Communication Mechanisms

- Edge nodes forward filtered events to the API Gateway over HTTPS/REST; the gateway authenticates the request and forwards it to the Message Broker.
- The Message Broker (Kafka/MQTT) implements asynchronous publish/subscribe messaging so multiple microservices can consume the same event stream independently.
- Microservices communicate with each other using lightweight REST/gRPC calls for synchronous needs (e.g., Evacuation Routing querying the Geospatial DB) and asynchronous events for state changes (e.g., Prediction publishing a “risk-threshold-exceeded” event that triggers Alert Management).
- The Alert Management Service fans a single alert out to multiple external gateways (SMS/IVR, push, siren, social media) in parallel for maximum reach and redundancy.
- Client applications receive updates via push notifications and periodic/streaming API calls to the dashboard backend, and can send citizen reports back into the ingestion pipeline.

4.3 Overall Workflow

(1) Sensors and external feeds continuously stream data to nearby edge nodes. (2) Edge nodes filter noise and forward only meaningful events to the API Gateway, which publishes them onto the event bus. (3) The Prediction & Risk Analysis Service consumes these events, runs its models and computes a risk score. (4) If the risk score crosses a threshold, it publishes a trigger event. (5) The Alert Management Service consumes the trigger, composes the alert and pushes it simultaneously through all dissemination channels while also invoking the Evacuation Routing Service. (6) The Evacuation Routing Service computes safe routes using live geospatial and hazard data and returns them to citizen apps. (7) The Shelter & Resource Management Service updates capacity and resource allocation as citizens move to shelters. (8) The Authority Dashboard aggregates data from all services in real time so officials can monitor and, if needed, override or escalate the response, coordinating with external emergency services.

5. Discuss Quality Attributes

5.1 Scalability

Each microservice can be scaled horizontally and independently based on load; the message broker buffers bursts of sensor/alert events so consumers are not overwhelmed during sudden demand spikes. Auto-scaling policies (e.g., scale the Alert Management and Evacuation Routing services first) can be applied selectively where load is expected to be highest during an active disaster, rather than scaling the entire system uniformly.

5.2 Security

The API Gateway centralises authentication, authorization and rate-limiting for all inbound traffic, preventing spoofed sensor data or forged alert triggers. Sensitive data (citizen location, personal details) is encrypted in transit (TLS) and at rest. Role-based access control restricts dashboard actions to authorised personnel, and inter-service calls within the microservices layer are authenticated (e.g., mutual TLS or signed tokens) to prevent lateral compromise.

5.3 Performance

Asynchronous, event-driven processing avoids blocking calls between services, keeping the detection-to-alert path short. Edge pre-processing reduces the volume of data that must travel to the cloud, cutting latency at the source. Caching of frequently accessed data (e.g., current shelter capacity, cached map tiles) and CDN delivery of the citizen app's static content further reduce response times.

5.4 Reliability

The event-driven design with a durable message broker ensures events are not lost even if a downstream consumer is temporarily unavailable — the broker retains and redelivers messages. Retry policies and circuit breakers between services prevent a slow or failing dependency from cascading. Redundant delivery across multiple alert channels (SMS, push, siren, social media) means a citizen is very likely to receive at least one alert even if one channel fails.

5.5 Maintainability

Each microservice owns its own codebase, data store and deployment pipeline, so a change to, say, the Prediction model does not require redeploying the Evacuation Routing service. Well-defined API/event contracts between services allow teams to evolve components independently, and the internal layered structure of each service keeps business logic, data access and presentation concerns separated for easier testing and debugging.

5.6 Availability

Services are deployed across multiple availability zones/regions with automatic failover, so regional infrastructure damage from the very disaster being monitored does not take the whole system offline. The citizen mobile app caches the last known alert and route information for offline access when connectivity is disrupted. Health checks and orchestration (e.g., Kubernetes) automatically restart or reroute traffic away from failed service instances.

6. Justify Architectural Decisions

6.1 Rationale

The hybrid microservices plus event-driven approach was chosen because the problem domain is inherently reactive (respond to unpredictable real-world events), highly variable in load, and life-critical, so no single component may become a bottleneck or single point of failure. Decoupling services through an event bus mirrors the natural “detect → predict → alert → respond” workflow of disaster management, while allowing each stage to be engineered, tested and scaled on its own terms.

6.2 Trade-offs Considered

- **Operational complexity vs. resilience:** Microservices introduce deployment, monitoring and inter-service-communication overhead compared to a monolith, but this complexity is accepted because independent scaling and fault isolation are essential for a life-safety system.
- **Eventual consistency vs. strict consistency:** The event-driven design favours eventual consistency (e.g., a slight delay before the dashboard reflects the latest shelter count) in exchange for higher throughput and availability; this is acceptable because most disaster-management data does not require strict ACID transactions across services.
- **Infrastructure cost vs. redundancy:** Multi-region deployment and multi-channel alerting increase infrastructure cost, but the trade-off favours redundancy given the life-safety consequences of a missed alert.
- **Development speed vs. flexibility:** A simpler layered monolith would have been faster to build initially, but would not meet the scalability and fault-isolation requirements identified in Section 1, so the additional upfront design effort for microservices was judged worthwhile.

6.3 Support for Future Enhancements and Long-Term Maintainability

The chosen architecture supports future growth in several ways:

- New disaster types or data sources (e.g., additional sensor networks, satellite feeds) can be onboarded by adding new producers to the event bus without modifying existing services.
- New alert channels (e.g., smart-home integrations, wearable alerts) can be added as new subscribers to the Alert Management Service's trigger events.
- Prediction models can be upgraded or replaced independently as better ML techniques become available, since the Prediction service is isolated behind a stable event/API contract.
- The API-first, loosely-coupled design simplifies integration with additional government or NGO systems as partnerships expand.
- Because each service is independently deployable, the system supports continuous integration/continuous deployment (CI/CD), enabling frequent, low-risk updates without full-system downtime.

Overall, the hybrid microservices and event-driven architecture directly addresses the functional needs and the critical non-functional requirements — scalability, availability, reliability, performance, security and maintainability — identified in the requirements analysis, while remaining extensible for the system's long-term evolution.